
Informe de Alcance

Low Accuracy Kinda Intelligent Tracking Unit (LAKITU)



Integrantes:

Agustín Cayrus

Sofía Dutra

Ignacio Moreno

Santiago Peña

Fecha:

16 de Marzo de 2026

Índice

Índice.....	1
Introducción.....	3
Definición de Problema.....	4
Alcance.....	5
Requerimientos funcionales.....	5
Raspberry Pi.....	5
Backend, AWS.....	5
Frontend.....	6
Requerimientos no funcionales.....	6
Rendimiento.....	6
Disponibilidad.....	6
Escalabilidad.....	6
Seguridad.....	7
Usabilidad.....	7
Mantenibilidad.....	7
Nice to have.....	7
Requerimientos excluidos del alcance.....	7
Materiales.....	8
Hardware.....	8
Software.....	8
Servicios externos.....	8
Protocolos y medios de comunicación.....	8
Arquitectura de la solución.....	9
Diagrama de componentes.....	10
Diagrama de despliegue.....	11
Metodología de trabajo.....	12
Planificación de Sprints.....	12
Sprint 1 (Semana 4-5).....	12
Sprint 2 (Semana 6-7).....	13
Sprint 3 (Semana 8-9).....	13
Sprint 4 (Semana 10-11).....	13
Sprint 5 (Semana 12-13).....	13
Sprint 6 (Semana 14-15).....	14

Introducción

La gestión y búsqueda de lugares de estacionamiento, particularmente en establecimientos muy asistidos, es una actividad tediosa que puede llegar a generar sentimientos de frustración con los que cualquier conductor puede sentirse identificado. Hoy en día las instituciones cuentan con la tecnología y los recursos necesarios para proveer solución a este problema universal. Nosotros presentaremos una posible solución al susodicho problema.

Mediante el uso de herramientas y técnicas modernas, como computer vision, edge computing, y cloud computing, es posible diseñar un sistema capaz de presentar, en tiempo real, la disponibilidad y ubicación de los lugares en un estacionamiento dado. Dicho sistema debe ser escalable, usable, y de altos rendimiento y disponibilidad.

Nuestra solución, a alto nivel, procesará en tiempo real las imágenes captadas por una cámara de manera local, para luego enviar los datos obtenidos a un backend presente en la nube. Dicha información podrá ser consultada en tiempo real y, adicionalmente, será almacenada de forma persistente con el fin de permitir el acceso a datos históricos y el cálculo de estadísticas relevantes por parte de los usuarios autorizados.

Con este proyecto se espera aprender a utilizar nuevas tecnologías, como lo son el procesamiento de información en un Raspberry Pi, procesamiento de imágenes en tiempo real mediante computer vision, entre otros. Adicionalmente se trabajará en un ambiente lo más similar posible a uno de producción real, empleando prácticas y metodologías de trabajo modernas y populares.

A continuación se plantea la definición del problema en la sección 2. La sección 3 entra en gran detalle sobre el alcance de nuestra solución, mientras que la sección 4 simplemente lista los materiales a necesitar. Finalmente se presentará la arquitectura diseñada y se explicará nuestra metodología de trabajo en las secciones 5 y 6 respectivamente.

Definición de Problema

Hoy en día no es inusual que los estacionamientos con sistemas de conteo y señalización de lugares disponibles aún requieran cierta participación humana frecuente; o, por el contrario, demanden de enormes y complejas infraestructuras. La primera situación — intervención frecuente — puede llevar a la presencia de errores humanos, falta de disponibilidad en ciertos horarios, o un alto coste de personal; mientras que la segunda claramente conlleva un superior coste de instalación y mantenimiento, y no resulta accesible para instituciones de una escala media o baja.

Nuestra solución no planea afrontar el problema general; en su defecto, la consideramos ideal para un estacionamiento de escala media, perteneciente a una institución que no puede permitirse la instalación de un sensor en cada lugar ofrecido. Esta es exactamente la situación que enfrenta el Edificio Parque de Innovación del LATU de la Universidad de Montevideo, al que asistimos, y en el que planeamos probar el sistema.

Se identifica entonces la necesidad de una solución que, prescindiendo de sensores físicos por lugar de estacionamiento y de intervención humana continua, sea capaz de determinar en tiempo real la disponibilidad de lugares en un estacionamiento de escala media, y de exponer esa información a sus usuarios. La ausencia de tal solución en el estacionamiento de nuestra facultad constituye el problema que buscamos resolver.

Alcance

A partir del problema identificado, esta sección delimita el alcance del sistema, definiendo las funcionalidades que se desarrollarán y las restricciones bajo las cuales operará.

Requerimientos funcionales

Raspberry Pi

- **RF-01 — Procesamiento de video:** Procesa el video capturado por la cámara en tiempo real, registrando los lugares libres y sus ubicaciones. A su vez se planea monitorear el comportamiento de los vehículos para inferir y detectar disponibilidad de lugares a los que no haya línea de visión directa. Se asume un punto de vista fijo, dado y definido en el desarrollo, del cual la cámara captura todo su contenido.
- **RF-02 — Identificación consistente de lugares:** A cada lugar de estacionamiento se le asociará cierta id, y esta debe ser consistente en todo el sistema, de manera que la disposición de los mismos siempre pueda ser aplicado a un cierto mapeo. De esta manera resulta trivial asociar ciertos valores de id a espacios reservados o espacios para discapacitados, y así mostrarlos con distinción.
- **RF-03 — Transmisión de cambios de estado:** Envía los datos obtenidos a un servidor en Amazon Web Services únicamente cuando detecte un cambio en el estado actual del estacionamiento (un lugar se libera o se ocupa). A esta transmisión también se le agrega una fotografía y un timestamp del instante en el que se detectó el cambio de estado.
- **RF-04 — Fotografía a demanda:** Saca y envía fotografías a demanda del backend.

Backend, AWS

- **RF-05 — Distribución del estado actual:** Redirige los datos más recientes del estado del estacionamiento inmediatamente a aquellos usuarios que los soliciten.

- **RF-06 — Almacenamiento persistente:** Una base de datos relacional SQL almacena lo siguiente:
 - Credenciales de usuarios finales y administradores, con las respectivas medidas de seguridad que esto conlleva.
 - Histórico del estado del estacionamiento: los datos de cada lugar, junto con la foto y el timestamp asociados.
- **RF-07 — Autenticación:** Realiza la validación y autenticación de los usuarios.
- **RF-08 — Reconstrucción de estado histórico:** Al recibir una consulta histórica de un momento arbitrario, deberá inferir el estado del estacionamiento en dicho instante, el cual será el almacenado en el último registro anterior a dicho instante.
- **RF-09 — Cálculo de estadísticas:** Calcula estadísticas relevantes con los datos obtenidos y almacenados.
- **RF-10 — Solicitud de fotografía:** Solicita al Raspberry Pi una fotografía actual.

Frontend

- **RF-11 — Visualización:** Representa la información recibida en un mapa 2D del estacionamiento, claramente mostrando los espacios libres y ocupados.
- **RF-12 — Interfaz de administrador:** Interfaz gráfica para los administradores, donde podrán solicitar los datos almacenados en el servidor de AWS.
- **RF-13 — Dashboard de estadísticas:** Dashboard para visualizar las estadísticas.

Requerimientos no funcionales

Rendimiento

- Toda la información capturada por la cámara deberá ser procesada en tiempo real por el Raspberry Pi, con una latencia máxima de 3 segundos entre un cambio físico y su manifestación en el sistema.
- Al solicitar una foto actual la latencia deberá ser menor a 1 minuto.
- Al solicitar datos históricos la latencia deberá ser menor a 30 segundos

Disponibilidad

- El sistema debe estar disponible al menos el 99% del tiempo durante el horario de funcionamiento del estacionamiento.
- Ante una pérdida de conexión el sistema debe seguir funcionando de manera local, y se deberán actualizar los datos del backend tan pronto como se logre la reconexión.

Escalabilidad

- El sistema debe facilitar la adición de cámaras para modelar nuevas secciones del estacionamiento, u otros estacionamientos.
- Se debe poder soportar múltiples solicitudes simultáneas.

Seguridad

- El acceso a las distintas funciones del sistema requerirá autenticación y validación de permisos.
- Las contraseñas deberán ser almacenadas de manera segura.

Usabilidad

- La interfaz debe ser intuitiva, tanto para usuarios finales como para los administradores.

Mantenibilidad

- El sistema debe estar bien documentado.
- Se deberá contar con un módulo de logs para fácilmente identificar y corregir errores.

Nice to have

- Una pequeña pantalla capaz de presentar los datos y el estado del estacionamiento de manera local, con un frontend básico ejecutado totalmente en el Raspberry Pi.
- Detectar cuando un vehículo ocupa más de un lugar de estacionamiento al mismo tiempo (y reportarlo).
- Medir la velocidad de los vehículos dentro del estacionamiento y registrar si se detecta un excedente del límite permitido.

Requerimientos excluidos del alcance

- Adaptación a imágenes del estacionamiento tomadas desde ángulos y posiciones no planificadas durante el desarrollo. Se limitará a un único punto de vista de video.
- Pagos o barreras de acceso físico propias.
- Gestión de reservas o pre-asignación de lugares.
- Integración con sistemas de control de acceso del edificio.
- Gestión de múltiples estacionamientos desde una misma instancia del sistema, más allá de la escalabilidad contemplada en los requerimientos no funcionales.
- Identificar y registrar las matrículas de los autos cuando sea posible.

Materiales

Hardware

1. Raspberry Pi 5.
2. Cámara compatible con un Raspberry Pi 5.
3. 7 Inch IPS DSI Display compatible con Raspberry Pi 5 (Nice to have).
4. Red WiFi del edificio.

Software

1. Python.
2. Librería de computer vision, OpenCV.
3. Framework de backend, FastAPI.
4. Typescript.
5. Framework del frontend, Next.js.
6. PostgreSQL.

Servicios externos

1. AWS IoT Core.
2. AWS SQS
3. AWS elasticache
4. AWS S3

Protocolos y medios de comunicación

1. MQTT.
2. HTTPS, REST.
3. Websockets.

Arquitectura de la solución

Dadas la complejidad moderada y escala media del proyecto, nuestra solución no exige una arquitectura particularmente avanzada, o que intente abordar problemas de acoplamiento y escalabilidad en su diseño intrínseco. Por estas razones decidimos descartar arquitecturas orientadas a servicios, y en su lugar emplear una arquitectura basada en capas.

Nuestra arquitectura cuenta con 5 capas:

- Capa de *presentación*, donde se hospeda a todos los elementos de frontend y UI con los que el usuario final, ya sea cliente o administrador, interactúa.
- Capa de *ingesta de datos*, la cual es el punto de entrada hacia el backend, donde toda la información externa es procesada, validada, y redireccionada al componente adecuado para su correcto uso en el sistema.
- Capa de *lógica de negocio*, donde ocurre toda la manipulación y donde se toman las decisiones de almacenamiento de los datos.
- Capa de *persistencia de datos*, necesaria para centralizar las comunicaciones con los centros de datos y abstraer a las capas superiores de la implementación de los mismos.
- Capa de *datos*, donde se tienen tanto un datalake S3 para las imágenes, como una base de datos relacional para la información estructurada.

En cuanto a hardware, nuestro sistema se ejecuta en 3 — o 4, siendo estrictamente técnicos — dispositivos diferentes. Los esenciales a mencionar son el Raspberry Pi, donde los datos relevantes del estacionamiento son recolectados, el servidor AWS donde son procesados, y el dispositivo cliente que los consume. En cuanto al frontend provisto a los clientes, se trata de una aplicación web de next.js, naturalmente hosteada en los servidores de Vercel, la distribuidora principal de next.js, el cuarto dispositivo a mencionar.

Dados los requisitos no funcionales de rendimiento y disponibilidad, especialmente en cuanto a la función de visualización en tiempo real del estado del estacionamiento y de la captura de fotografías a comando, debe existir un medio de comunicación bidireccional

en tiempo real entre el cliente y el backend, lo que naturalmente lleva a la decisión de websockets. Adicionalmente, para acelerar más aún el proceso, se cuenta con un caché dedicado al último estado registrado del estacionamiento. Para todas las comunicaciones, que no demandan tal instantaneidad, una simple API REST es suficiente.

Finalmente, para la comunicación entre el Raspberry Pi y el backend, decidimos delegar la complejidad a servicios provistos por AWS, como lo son IoT Core — servicio capaz de autenticar dispositivos IoT, pensado para la comunicación basada en eventos con dichos dispositivos — y una cola de eventos gestionada con SQS, la cual sirve de buffer en caso de que el backend no esté funcionando cuando IoT Core intenta la comunicación.

Diagrama de componentes

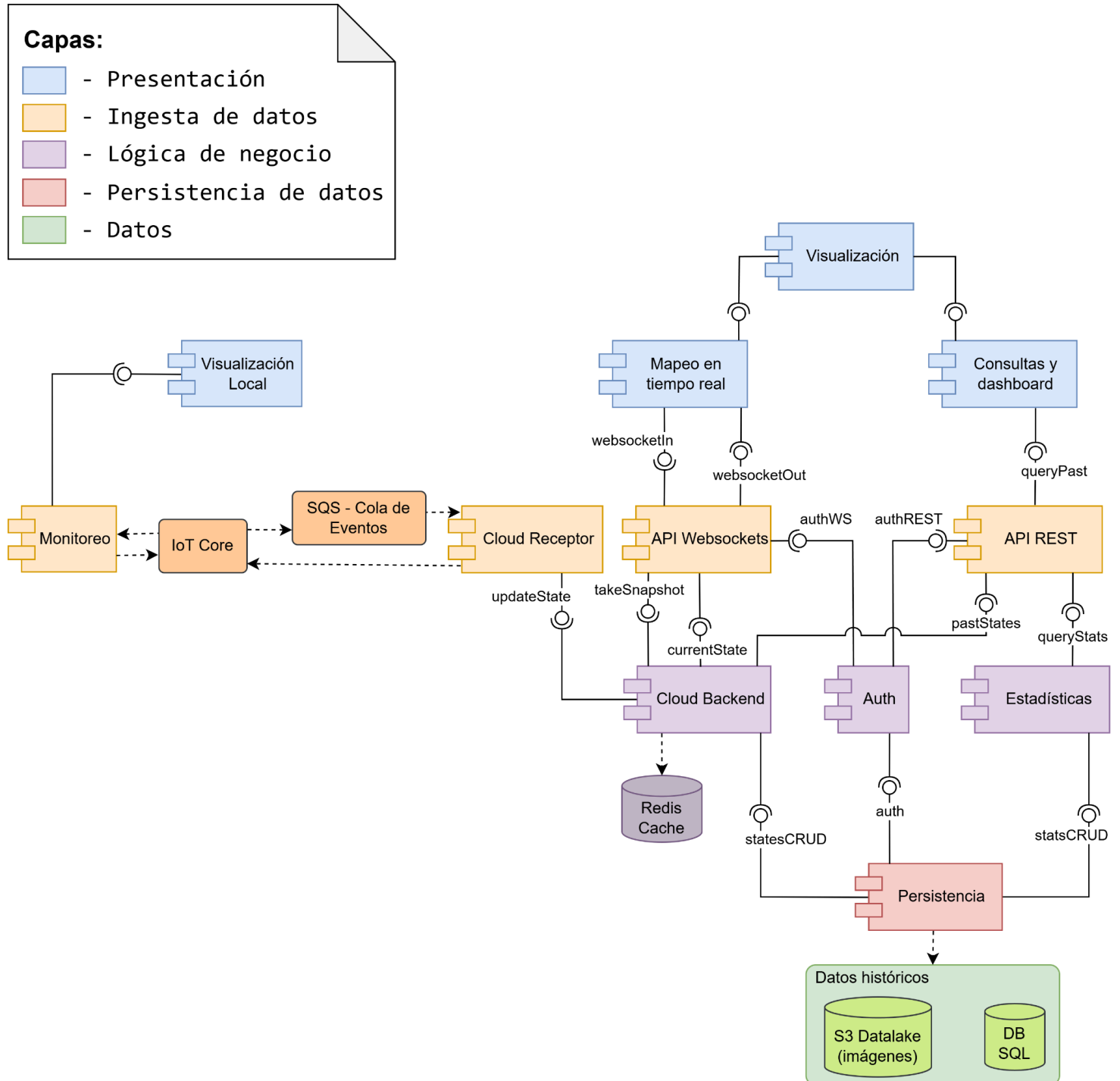
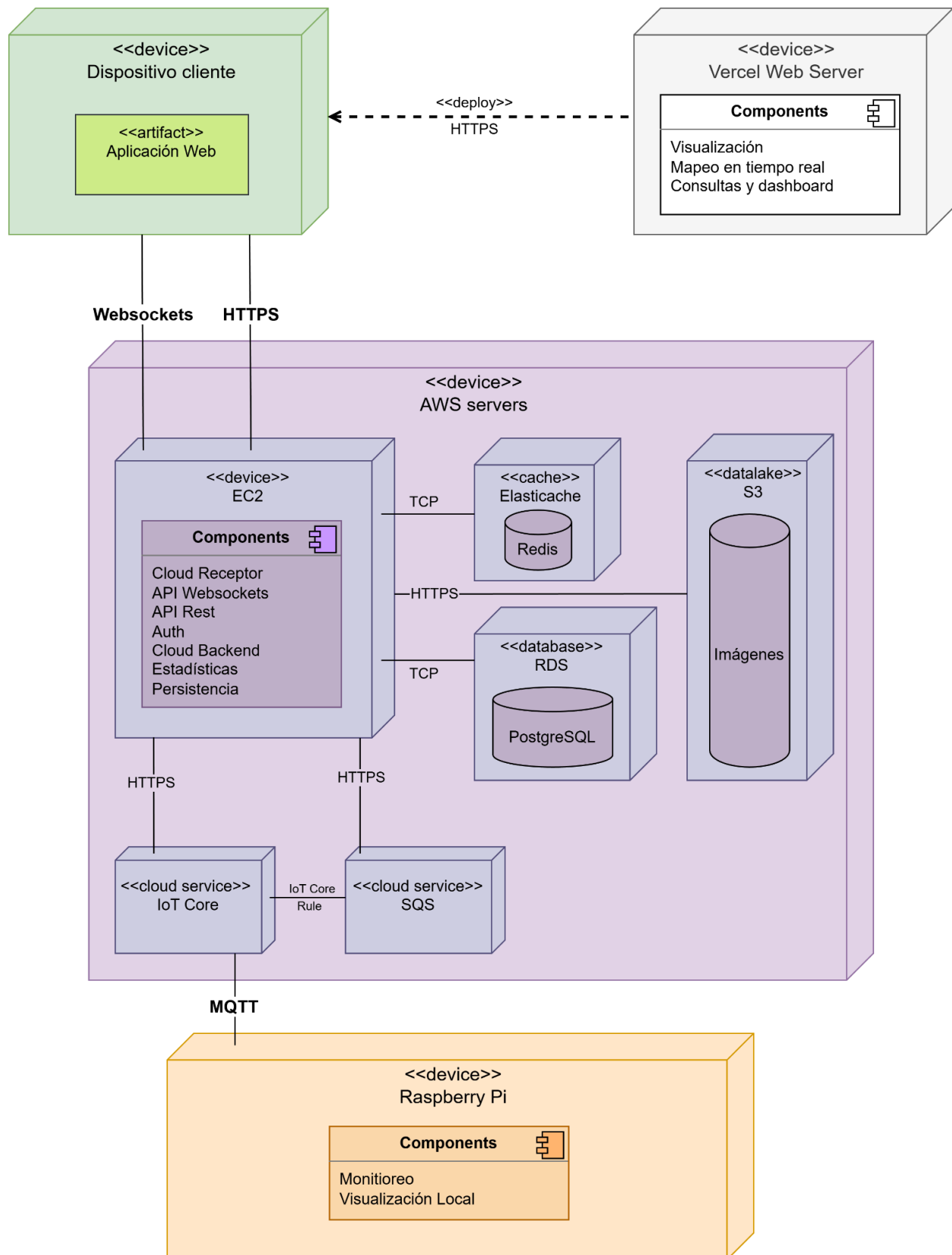


Diagrama de despliegue



Metodología de trabajo

Para el proyecto se emplea el uso de metodologías ágiles, particularmente con el uso de SCRUM en sprints de 2 semanas, gestionado con Microsoft Planner. Durante dichos sprints se asignan tareas y roles específicos a los integrantes del equipo, donde se espera un seguimiento de cierto rol más general de cada integrante, uno que abarque toda la duración del proyecto. Cada tarea necesaria para el cumplimiento del MVP será especificada y detallada dentro del product backlog, y puntos serán otorgados en base a su complejidad y tamaño.

El equipo se reunirá para discutir las tareas necesarias y se usará poker planning y valores de fibonacci para calcular la distribución de puntos. En base al progreso del proyecto, y la disponibilidad de cada integrante, se asignan tareas teniendo en cuenta los puntos asociados. Al final de cada sprint se hará un sprint review, donde se revisa el resultado del sprint y el desempeño del grupo, de donde surge un sprint retrospective para discutir qué funcionó, qué no funcionó, y qué cambios adoptar para el siguiente sprint antes de iniciarlo.

Mediante el uso del software de control de versiones Git se tendrán ramas dedicadas a cada funcionalidad emergente. Durante todo el proceso se harán pruebas de humo para garantizar la funcionalidad básica. Tareas se consideran "Done" cuando funcionen y pasen la pruebas de humo en sus ramas locales, actualizadas con los últimos cambios de la rama principal. Al final de cada sprint se actualizará la rama principal con todos los cambios hechos en ramas locales, con la previa realización de "Pull requests" y aprobaciones mediante "Reviews" por parte de los demás integrantes del grupo.

Planificación de Sprints

A continuación se presenta la planificación general del proyecto, con los sprints y sus correspondientes tareas.

Sprint 1 (Semana 4-5)

- Configurar repositorio GitHub

- Establecer contrato AWS
- Establecer contrato Backend
- Configuración Inicial Backend
- Configuración Inicial Raspberry Pi
- Investigar Conexión Backend y Base de Datos
- Investigar Conexión Ras-Pi con AWS
- Documentación Decisiones de Arquitectura
- Documentación Sprint 1

Sprint 2 (Semana 6-7)

- Presentación Demo 30%
- Configuración Autenticación Usuarios
- Establecer contrato Frontend
- Conexión con Servidor AWS y Backend
- Conexión Raspberry Pi con Servidor AWS
- Configurar Servidor AWS
- Funcionalidad Monitoreo con Raspberry Pi y Cámara
- Funcionalidad Básica Backend
- Investigar Entornos de Testing
- Tests de Humo
- Documentación Sprint 2

Sprint 3 (Semana 8-9)

- Componente Estadísticas
- Funcionalidad Monitoreo Avanzada
- Configuración Inicial Frontend
- Tests de Humo
- Documentación Sprint 3

Sprint 4 (Semana 10-11)

- Presentación Demo 60%
- Funcionalidad Monitoreo (Nice to Have)
- Funcionalidad Básica Frontend
- Funcionalidad Avanzada Backend
- Pre-Informe
- Documentación Sprint 4

Sprint 5 (Semana 12-13)

- Mejora de Rendimiento de Monitoreo
- Análisis de Eficiencia de Monitoreo
- Funcionalidad Avanzada Frontend
- Documentación Sprint 5
- Finalizar Funcionalidades Backend

Sprint 6 (Semana 14-15)

- Entrega Final
- Documentación Sprint 6
- Stress testing
- Finalizar Funcionalidades Frontend